Лекция 5. Память программы на C++. Семантика перемещения

1. Память программы (процесса)
   - Стек
   - Куча
   - Сегмент BSS данных (Block Started by Symbol)
   - Сегмент данных (Data Segment)
   - Сегмент данных для чтения (RODATA Segment)
   - Сегмент кода программы (Text Segment)
   - Сегмент локальных переменных потока (Thread Local Segment)
2. Продолжительность хранения (*storage duration*)
   - Static storage duration
   - Automatic storage duration
   - Dynamic storage duration
   - Thread storage duration
3. Связывание (*linkage*)
   - Внешнее связывание (*external linkage*)
   - Внутреннее связывание (*internal linkage*)
   - Отсутствие связывания (*no linkage*)
   - Модульное связывание (*module linkage*) C++20
4. Ключевое слово `static`
   - Вне класса. Глобальная статическая переменная
   - Вне класса. Локальная статическая переменная
   - Вне класса. Глобальная статическая функция
   - Внутри класса. Статическое поле
   - Внутри класса. Статический метод
5. Ключевое слово `inline`
6. Initialization Order Fiasco
7. Вычисления на этапе компиляции
   - Константное выражение
   - Переменные `constexpr` (*C++11*)
   - Хранение `constexpr` переменных в памяти
   - Функции `constexpr` (*C++11*)
   - Динамическая память в `constexpr` (*C++20*)
   - Функции `consteval` (*C++20*)
   - Ключевое слово `constinit` (*C++20*)
   - `if consteval` (*C++23*)
8. Работа с динамической памятью
   - Операторы `new` и `delete`
   - Операторы `new[]` и `delete[]`

- Переопределение операторов `new` и `delete`

9. Ключевое слово `noexcept`
10. Категория выражений (value categories)
    - rvalue-ссылка
    - Связывание ссылок
    - Перегрузка функции по ссылке
    - Квалификация метода по виду ссылки (*C++11*)
    - Temporary materialization (C++17)
11. Семантика перемещения
    - Функция `std::move`
    - Правило нуля (*Rule of Zero*)
    - Правило пяти (*Rule of Five*)
    - Правила для специальных методов
12. Copy Elision
    - Return Value Optimization (RVO)
    - Named Return Value Optimization (NRVO)

Память программы (процесса)

Программа имеет виртуальное адресное пространство. При запуске программа не полностью загружается в оперативную память. Операционная система (ОС) настраивает таблицы страниц и выделяет области памяти для программы. Таблицы страниц служат для отображения виртуальных адресов программы на C++ в физические адреса оперативной памяти.

В языке C++ упрощенно можно выделить следующие сегменты памяти:

- **автоматический** - представлен стеком, служит для автоматического выделения и освобождения памяти под локальные переменные, аргументы функции.
- **статический** - для глобальных и статических переменных, существует всю жизнь программы
- **динамический** - представлен кучей, позволяет получать память в использование во время выполнения программы и требует управления выделением и освобождением памяти
- **потоковый** - для специально помеченных локальных переменных потока, существует всю жизнь потока

Память процесса можно разделить на следующие сегменты:

- Stack (Стек)
- TLS (Thread local segment)
- Heap (Куча)
- BSS (Block Started by Symbol)
- Data Segment
- RODATA Segment (read only)

- Text Segment (Code)

Стек

Стек имеет ограниченный размер, который зависит от ОС и её настроек. По умолчанию размер составляет 1 МБ в Windows и 8 МБ в Linux (`ulimit -s`). Стек организован по принципу LIFO.

Выделяемая память на стеке **НЕ** инициализируется по умолчанию. Поэтому локальные переменные функций без инициализатора содержат неопределённые значения, а их чтение - **UB**.

При вызове функции на стеке формируется стек фрейм, в котором размещаются параметры функции, локальные переменные, адрес возврата, сохраненные регистры.

Компилятор **НЕ** гарантирует, что переменные на стеке будут лежать в порядке объявления.

При выходе из области видимости вызываются деструкторы объектов в порядке, обратном созданию.

При выходе из функции указатель вершины стека (stack pointer, регистр `rsp` на x86-64) просто сдвигается обратно. Тем самым память автоматически считается свободной.

Переполнение стека приводит к ошибке stack overflow. В Linux процесс получает сигнал `SIGSEGV` (segmentation fault), в Windows — исключение `STATUS_STACK_OVERFLOW` .

К переполнению может приводить как размещение слишком больших массивов, так и глубокий рекурсивный вызов функций.

Куча

Динамическая память (куча) представляет собой участок памяти, требующий ручного управления. Динамическая память выделяется во время выполнения программы и имеет продолжительность хранения, пока не будет освобождена.

Выделение памяти в куче медленнее стека, поскольку при выделении может происходить запрос к менеджеру памяти ОС (системный вызов) во время выполнения программы. Также, аллокатор ищет свободный блок и ведёт учёт блоков, в многопоточной программе синхронизирует доступ. Тогда как выделение на стеке — это один сдвиг stack pointer.

Сегмент BSS данных (Block Started by Symbol)

Данный сегмент хранит глобальные и статические переменные, которые не имеют явной инициализации или инициализированы нулем.

Загрузчик ОС при запуске программы выделяет память под эти переменные и зануляет её, поэтому они не занимают места в бинарном файле, что позволяет экономить место.

Сегмент данных (Data Segment)

Данный сегмент хранит глобальные и статические переменные, которые явно инициализированы ненулевыми значениями. Переменные занимают место в бинарном файле.

Сегмент данных для чтения (RODATA Segment)

Как следует из названия, в данном сегменте лежат данные для чтения. Поэтому попытка изменения данных приводит к ошибкам компиляции или к UB, падению программы при попытке изменить константу. Продолжительность хранения объектов данного сегмента статическая - все время жизни программы.

В данном сегменте лежат именованные константы, инициализированные константным выражением, и строковые литералы.

Строковые литералы могут лежать в единственном экземпляре, но это не гарантируется.

Сегмент кода программы (Text Segment)

В данном сегменте располагается машинный код программы. В данном сегменте записаны функции, методы и точка входа в программу `main`. Как правило, данный сегмент предназначен только для чтения и является общим для всех процессов, запущенных из одного исполняемого файла.

Сегмент локальных переменных потока (Thread Local Segment)

В данном сегменте располагаются локальные переменные потока. Для таких переменных при объявлении перед типом используется ключевое слово `thread_local` (*C++11*) — спецификатор класса хранения. У каждого потока свои копии переменных, а их время жизни соответствует времени жизни потока.

Данные лежат в секциях `.tdata` (с инициализатором) и `.tbss` (нулевые).

Продолжительность хранения (*storage duration*)

В языке C++ с сегментами памяти тесно связано понятие storage duration.

Продолжительность хранения (storage duration) - период, в течение которого зарезервирована память для объекта.

Продолжительность хранения - это свойство объекта, которое определяет минимальный потенциальный срок жизни выделенной памяти под объект.

В языке C++ выделены следующие виды продолжительности хранения:

- static
- automatic
- dynamic
- thread

Static storage duration

Static storage duration - память выделяется и освобождается независимо от области видимости объекта и продолжает существовать все время выполнения программы.

Соответствует сегментам памяти `.bss`, `.data`, `.rodata`.

Глобальные, статические переменные, в том числе объявленные внутри `namespace`, имеют static storage duration.

Инициализируется до входа в `main` (для локальных статических переменных - когда выполнение впервые доходит до соответствующего объявления).

Порядок инициализации между единицами трансляции **НЕ** определен. Следовательно, некорректно инициализировать переменную со статическим storage duration, переменной с аналогичным storage duration из другой единицы трансляции (файла).

```
int global;
int global_init = 18;
static int static_value = 10;
extern int value;

namespace utils {
    const double IN_TO_CM = 2.54;
}

int& get_static() {
    static int instance;
    return &instance;
}
```

Automatic storage duration

Automatic storage duration - память выделяется при входе в область видимости и освобождается при выходе из области видимости, причем вызываются деструкторы.

Представлен стеком.

```
void function(int a, double b, const char* name) {
    std::string info(name);
    {
        double num = a * b;
        info += std::to_string(num);
    }
    std::cout << info << std::endl;
}
```

- переменные `a`, `b`, `name`, `info`, `num` имеют automatic storage duration

---

Dynamic storage duration

Dynamic storage duration - управление памятью осуществляется вручную.

Представлен кучей

```
int* ptr = new int(5);
int* arr = new int[5]{1, 2, 3};
delete ptr;
delete[] arr;
```

Thread storage duration

Thread storage duration - память выделяется при запуске потока и освобождается при его завершении.

У каждого потока своя копия

```
thread_local int value = 18;
```

Связывание (*linkage*)

Связывание определяет видимость идентификаторов между единицами трансляции. Связывание не относится к памяти напрямую, а затрагивает разрешение имён на этапе линковки.

Внешнее связывание (*external linkage*)

External linkage - внешнее связывание означает, что идентификатор (имя) виден в других единицах трансляции. По умолчанию для функций и переменных в глобальной области видимости и в `namespace` (кроме `const`, `constexpr` переменных и сущностей безымянного `namespace`).

Внутреннее связывание (*internal linkage*)

Internal linkage - внутреннее связывание, означает, что идентификатор (имя) виден только в текущей единице трансляции. Для `static` переменных и функций, `const` и `constexpr` переменных (если они **НЕ** `extern` и **НЕ** `inline`), всего, что объявлено в безымянном `namespace`.

Таким образом `const` переменная в заголовочном файле, включаемом в разные `.cpp` файлы, представлена **копией** в каждой единице трансляции.

Для внешнего связывания `const` переменной используется ключевое слово `extern`.

В *C++17* для константы в заголовочном файле используется `inline constexpr`, что позволяет получить одно определение с внешним связыванием на всю программу. Тогда как для `constexpr` переменной адрес будет уникальный в каждой единице трансляции.

Отсутствие связывания (*no linkage*)

No linkage - отсутствует связывание, означает, что идентификатор (имя) виден только в области видимости. Для параметров функций, имен шаблонных параметров, локальных переменных.

Модульное связывание (*module linkage*) C++20

В языке C++20 появились модули `module` и вместе с ними появился новый вид связывания.

Module linkage (C++20) - модульное связывание означает, что идентификатор (имя) виден в пределах модуля и не виден извне.

Модуль может включать несколько единиц трансляции.

Ключевое слово `static`

Ключевое слово `static` применяется к идентификатору переменной или функции и имеет разные смыслы в зависимости от контекста использования.

Вне класса. Глобальная статическая переменная

Глобальная переменная, помеченная ключевым словом `static`, имеет статическую продолжительность хранения (static storage duration) и внутреннее связывание (internal linkage), то есть видна только в текущей единице трансляции. Что позволяет скрыть реализацию в .cpp файле и избежать конфликтов **ODR** (*One Definition Rule*).

```
static int value = 18;
```

НО если определить её в `.h` файле, то каждый включающий её `.cpp` файл будет иметь копию, видимую только в пределах данного файла.

Вне класса. Локальная статическая переменная

Локальная статическая переменная (внутри функции) имеет *static storage duration* и область видимости внутри функции, а также инициализируется только **один раз**, когда выполнение впервые доходит до объявления. Начиная с *C++11*, эта инициализация потокобезопасна.

```
void f() {
    static int counter = 0;
    ++counter;
    std::cout << "f() call count = " << counter << std::endl;
}
```

- **НЕ** видна вне функции

Как правило используется в качестве кэша, счетчика, синглтона (singleton), при ленивой инициализации (lazy initialization).

Вне класса. Глобальная статическая функция

Глобальная функция, помеченная ключевым словом `static`, также имеет внутреннее связывание и не видна из других единиц трансляции (`.cpp` файлов).

```
static void inner_helper();
```

В современном C++ заменяют `static` в данном контексте на размещение глобальной функции или переменной в безымянном пространстве имен `namespace`.

Внутри класса. Статическое поле

Переменная внутри класса, объявленная с ключевым словом `static`, имеет статическую продолжительность хранения (static storage duration) и то же связывание, что у класса, обычно

внешнее (external linkage).

Статическая переменная класса принадлежит классу, а не конкретному экземпляру.

Доступ к статическому полю принято осуществлять посредством имени класса и оператора `::`. Также доступно через экземпляр класса и оператор `.`

Статическое поле является объявлением. Требует определения только в одном `.cpp`, поэтому инициализация неконстантного поля в теле класса — ошибка компиляции.

```
// .h
class C {
    static int counter; // declaration
    // static int counter = 0; // compile error
};
// .cpp
int C::counter = 0; // definition
```

Инициализация `static` поля внутри класса разрешена для интегральных констант.

НО для получения адреса константного статического поля необходимо определение **ВНЕ** класса, даже если значение проинициализировано.

```
// .h
class C {
    static const int multiplier = 2; // declaration
    // static const double PI = 3.14; // error double is not integral
};
// .cpp
const int C::multiplier; // definition
const int* ptr = &C::multiplier; // work after definition
```

Инициализация `static` поля внутри класса разрешена для `constexpr` полей (*C++11*).

```
// .h
class C {
    static constexpr int multiplier = 2; // declaration
    static constexpr double PI = 3.141592653589793; //ok
};
// .cpp
constexpr int C::multiplier; // definition before C++17, deprecated
const int* ptr = &C::multiplier; // work after definition
```

- Но для получения адреса необходимо определение **ВНЕ** класса до C++17. Начиная с C++17 `constexpr` статическое поле неявно является `inline` и определение **ВНЕ** класса **НЕ** требуется

Инициализация `static` поля внутри класса разрешена для `inline static` полей (C++17). Является определением, определение **ВНЕ** класса **НЕ** требуется, разрешено в заголовке.

```cpp
class C {
    inline static int multiplier = 2;
    inline static std::string default_name = "name";
};
```

Внутри класса. Статический метод

Функция внутри класса, объявленная с ключевым словом `static`, не привязана к какому-либо экземпляру (объекту) класса, а логически принадлежит всему классу. Следовательно, внутри функции нет указателя на объект `this`.

```cpp
struct A {
    static int f();
};
```

Внутри функции можно обращаться только к `static` полям, к обычным напрямую нельзя, только через объект класса, принятый в аргументах (в том числе к приватным).

```cpp
struct A {
    static int field;
    static int f() { return field; } // OK
    static int f(const A& obj) { return obj.field_; } // OK
private:
    int field_;
};
int A::field = 0;
```

Вызвать функцию можно с указанием принадлежности классу, используя оператор `::`

```cpp
int value = A::f();
```

Ключевое слово `inline`

Ключевое слово `inline` означает, что можно иметь определение сущности, которое должно быть одинаковым, в нескольких единицах трансляции. Линковщик оставит одно определение. Благодаря `inline` можно определить функцию или переменную в заголовочном файле.

- `inline` функции (C++98)
- `inline` переменные (C++17)

```cpp
// .h
inline int square(int x) { return x * x; }
```

```cpp
inline int counter = 0;                          // C++17
inline constexpr double PI = 3.141592653589793; // C++17
```

- адрес `inline` функции или переменной одинаков во всех единицах трансляции для переменных и функций с внешним связыванием
- если определения различаются, то это нарушение ODR, при этом программа **НЕ** корректна, но компилятор и линковщик **НЕ** обязаны об этом сообщать

Неявно `inline` являются:

- методы, определенные внутри класса
- `constexpr` (C++11) и `consteval` (C++20) функции
- `constexpr static` поля класса (C++17)

ВАЖНО. `constexpr` переменная в `namespace` неявно `inline` **НЕ** является. Без `inline` каждая единица трансляции получает свою копию с внутренним связыванием.

ВАЖНО. `inline` **НЕ** означает, что компилятор встроит тело функции в место вызова. Встраивание (inlining) - оптимизация, которую компилятор выполняет по своему усмотрению для любых функций.

Использование `inline static` для полей класса позволяет объявить и определить поле прямо в классе (C++17). Без `inline` статическое поле, определяется вне класса.

Initialization Order Fiasco

Initialization Order Fiasco - ошибка, которая возникает, когда динамическая инициализация объекта со *static storage duration* использует другой такой объект из другой единицы трансляции. Порядок динамической инициализации между единицами трансляции **НЕ** определён, поэтому второй объект может быть ещё **НЕ** инициализирован.

В рамках одной единицы трансляции порядок инициализации определен сверху вниз.

```cpp
// a.cpp
extern std::string name;
std::size_t name_len = name.size(); // UB, если name ещё не создан

// b.cpp
std::string name = "a long string, longer than SSO buffer";
```

Результат зависит от порядка файлов при линковке:

```bash
g++ -std=c++20 b.cpp a.cpp main.cpp && ./a.out
# len = 37
g++ -std=c++20 a.cpp b.cpp main.cpp && ./a.out
# len = 0
```

- во втором случае `name.size()` вызывается для объекта, конструктор которого ещё не выполнялся, это **UB**
- в данной реализации `0` получился потому, что память `name` была занулена при static initialization, а внутреннее представление пустой `std::string` в libstdc++ дало `size() == 0`. Полагаться на это **нельзя**: вызов метода до начала времени жизни объекта — **UB**

Ошибка возникает **только** при динамической инициализации. Если значение, от которого зависит инициализация, получено в ходе константной инициализации (constant initialization), оно гарантированно готово до любой динамической инициализации. При этом сама зависимая переменная всё ещё может инициализироваться динамически.

```
// a.cpp
extern int y;
int x = y + 1; // всегда 11

// b.cpp
int y = 10;    // constant initialization
```

Пример с динамической инициализацией:

```
// a.cpp
extern int y;
int x = y + 1; // 11 или 1, зависит от порядка линковки

// b.cpp
int compute() { return std::atoi("10"); }
int y = compute(); // dynamic initialization
```

- во втором случае `x` может прочитать `y`, который пока только занулён, и получить `1`. Это **НЕ** UB для `int`, но результат не определён заранее

Один из способов избежать ошибки - инициализировать константным выражением и проверить это с помощью `constinit` (*C++20*).

Другой способ - это заменить глобальную переменную функцией с локальной статической переменной (*Construct On First Use*):

```
// b.cpp
const std::string& name() {
    static const std::string instance = "a long string, longer than SSO buffer";
    return instance;
}

// a.cpp
const std::string& name();
std::size_t name_len = name().size(); // ok, объект создаётся при первом вызове
```

- объекты со *static storage duration* уничтожаются в порядке, обратном созданию. Поэтому обратная ошибка (*destruction order fiasco*) возможна, если деструктор одного глобального

объекта обращается к другому

Вычисления на этапе компиляции

Часть значений программы известна уже во время компиляции. Если компилятор вычислит их сам, то в бинарный файл попадёт готовый результат: число прямо в инструкции или данные в сегменте `.rodata`. Тогда во время выполнения не нужен ни код вычисления, ни код инициализации.

Для управления вычислениями на этапе компиляции в C++ есть ключевые слова:

- `constexpr` (C++11)
- `consteval` (C++20)
- `constinit` (C++20)

Константное выражение

Константное выражение (*constant expression*) - выражение, которое компилятор может вычислить на этапе компиляции.

Константное выражение обязательно в следующих местах:

- размер встроенного массива и аргумент шаблона `std::array<int, N>`
- метка `case`
- условие `static_assert`
- значение перечислителя `enum`
- аргумент в `alignas()`

```
int read();

const int n = 10;
std::array<int, n> a1; // ok, value n is constant expression

const int m = read();
std::array<int, m> a2; // compile error, value m is known in runtime only
```

- `const` означает только нельзя изменять, а **НЕ** известно при компиляции
- `const` переменная интегрального типа, инициализированная константным выражением, может использоваться в константных выражениях (правило, оставшееся с C++98)

Известно на этапе компиляции:

- Литералы `(0, "hello", '\n', 1.0, 1ull)`
- Члены `enum`
- Параметры шаблонов

- Результат `sizeof` над типами
- Переменные `constexpr`

Переменные `constexpr` (*C++11*)

Спецификатор `constexpr` для переменной требует, чтобы она была инициализирована константным выражением. Иначе код не скомпилируется.

```
int read();

const int a = read();       // ok, const can be initialized in runtime
constexpr int b = read();   // compile error, read() is not constexpr
constexpr int c = 18;       // ok
constexpr double PI = 3.141592653589793; // ok
```

Сходства и различия `const` и `constexpr`:

- переменная `constexpr` **неявно** является `const`, следовательно изменять нельзя
- переменную `constexpr` можно инициализировать только константным выражением, во время компиляции, а `const` можно инициализировать также во время выполнения
- тип `constexpr` переменной должен быть *литеральным типом* (*literal type*): скалярные типы, ссылки, массивы литеральных типов, классы с `constexpr` конструктором и тривиальным или `constexpr` деструктором
- переменная `constexpr` может участвовать в константных выражениях, а переменная `const` только если интегральная с константным инициализатором
- переменная `constexpr` гарантирует вычисление при компиляции, `const` - **НЕТ**
- связывание у `const` и `constexpr` переменной в `namespace` - внутреннее

Хранение `constexpr` переменных в памяти

Спецификатор `constexpr` **НЕ** меняет продолжительность хранения переменной.

- `constexpr` в области видимости `namespace` имеет *static storage duration* и внутреннее связывание, как и `const`. Если адрес переменной не используется, компилятор подставляет значение прямо в код и память под переменную может вообще не выделяться. Если адрес нужен, объект размещается в `.rodata`
- локальная `constexpr` переменная имеет *automatic storage duration* и создаётся при каждом вызове функции.

Локальные массивы констант принято объявлять `static constexpr`:

```
int local_table(int i) {
    constexpr int table[8] = {1, 2, 4, 8, 16, 32, 64, 128};
    return table[i];
}
```

```cpp
int static_table(int i) {
    static constexpr int table[8] = {1, 2, 4, 8, 16, 32, 64, 128};
    return table[i];
}
```

- при `-O0` в `local_table` g++ при каждом вызове записывает 8 чисел на стек, clang копирует таблицу из `.rodata` на стек
- в `static_table` таблица одна, лежит в `.rodata`, функция только читает из неё
- при `-O2` компилятор часто сам выносит локальную таблицу в `.rodata`, но **НЕ** обязан: если адрес таблицы передаётся в другую функцию, копия на стеке остаётся и при `-O2`

Размещение переменных со *static storage duration* можно посмотреть утилитой `objdump`:

```cpp
int g_zero;                                          // .bss
int g_init = 18;                                     // .data
const int g_const = 5;                               // .rodata
constexpr int g_cexpr = 7;                           // .rodata
constexpr std::array<int, 4> g_table{1, 2, 3, 4};    // .rodata
std::array<int, 4> g_mut_table{1, 2, 3, 4};          // .data
const int g_runtime = std::atoi("5");                // .bss + dynamic init
const std::string g_str = "not a compile-time constant"; // .bss + dynamic init
constinit int g_ci = 42;                             // .data
thread_local int t_init = 1;                         // .tdata
thread_local int t_zero;                             // .tbss
```

- **ВАЖНО**: `const` объект **НЕ** обязательно лежит в `.rodata`. Если значение вычисляется во время выполнения, память лежит в `.bss` и заполняется кодом инициализации до `main`. Записать в такой объект всё равно нельзя (UB)
- `constexpr` гарантирует, что значение уже лежит в бинарном файле

Функции `constexpr` (*C++11*)

Функция `constexpr` может быть вычислена на этапе компиляции, если все её аргументы являются константными выражениями и результат нужен в константном выражении. Иначе она вызывается во время выполнения, как обычная функция.

```cpp
constexpr int factorial(int x) {
    int result = 1;
    for (int i = 2; i <= x; ++i) {
        result *= i;
    }
    return result;
}

static_assert(factorial(5) == 120); // compile time

int main(int argc, char**) {
    constexpr int f5 = factorial(5); // compile time
```

```
    int fn = factorial(argc);        // runtime
}
```

- `constexpr` функция неявно является `inline`, поэтому её определяют в заголовочном файле
- спецификатор `constexpr` - это разрешение, а **НЕ** требование вычислить при компиляции

Возможности `constexpr` функций расширялись от стандарта к стандарту:

- C++11 - тело функции из единственного `return`
- C++14 - локальные переменные, циклы, `if`, изменение локальных объектов
- C++17 - `constexpr` лямбды, `if constexpr`
- C++20 - `virtual` функции, `try` / `catch` (без выброса исключения), `new` / `delete`, `constexpr` деструкторы, `std::vector` и `std::string` в `constexpr` функциях
- C++23 - `static` и `thread_local` переменные внутри `constexpr` функций, `if consteval`

ВАЖНО: при вычислении на этапе компиляции компилятор обязан диагностировать **UB** уровня языка (выход за границу, переполнение, разыменование `nullptr`) Выражение, содержащее **UB**, **НЕ** является константным:

```
constexpr int at(int i) {
    int a[3]{};
    return a[i];
}
static_assert(at(2) == 0);         // ok
static_assert(at(3) == 0);         // compile error, выход за границу массива
constexpr int i_min = INT_MAX + 1; // compile error, переполнение int

int runtime = at(3);               // UB, но компилятор не обязан ничего
сообщать
```

- `static_assert` с вызовом `constexpr` функции - простой способ проверить код на отсутствие UB для конкретных входных данных

Динамическая память в `constexpr` (*C++20*)

Начиная с C++20 в `constexpr` функциях можно использовать `new` и `delete`. Условие использования: вся выделенная при константном вычислении память должна быть освобождена в рамках этого же вычисления (*transient allocation*).

```
constexpr int get_value() {
    int* ptr = new int(18);
    int value = *ptr;
    delete ptr;
    return value;
}
static_assert(get_value() == 18); // ok
```

```
constexpr int leak(int count) {
    int* ptr = new int[count];
    return ptr[0] = count;
}
static_assert(leak(1) == 1);      // compile error
```

По той же причине `std::vector` можно использовать внутри `constexpr` функции, но **НЕЛЬЗЯ** создать `constexpr` переменную типа `std::vector`:

```
constexpr int vec_sum() {
    std::vector<int> v{1, 2, 3};
    return std::accumulate(v.begin(), v.end(), 0);
}
static_assert(vec_sum() == 6);             // ok

constexpr std::vector<int> v{1, 2, 3};    // compile error
```

- в бинарном файле нет кучи, поэтому результат константного вычисления не может ссылаться на динамическую память
- аналогично для `std::string`

Функции `consteval` (*C++20*)

Функция `consteval` (*immediate function*) **обязана** вычисляться на этапе компиляции. Каждый её вызов должен быть константным выражением.

```
consteval int Square(int x) {
    return x * x;
}

int main(int argc, char**) {
    constexpr int a = Square(5); // ok
    int b = Square(10);          // ok
    int c = Square(argc);        // compile error
}
```

- `consteval` функция неявно `inline`
- `constexpr` - можно при компиляции, `consteval` - только при компиляции

Ключевое слово `constinit` (*C++20*)

Спецификатор `constinit` требует, чтобы переменная со *static* или *thread storage duration* прошла константную инициализацию. Если инициализация окажется динамической, будет ошибка компиляции.

В отличие от `constexpr`, переменная **НЕ** становится **константой** и её можно изменять.

```
constexpr int compute() { return 10; }
int read();

constinit int val = compute(); // ok, constant initialization
constinit int ce = read();     // compile error, read() is not constexpr

void f() {
    ++val;                     // ok, constinit is not const
    constinit int local = 0; // compile error, is not static or thread storage
duration
}
```

- `constinit` нельзя сочетать с `constexpr`, `constexpr` уже гарантирует константную инициализацию
- в другой единице трансляции можно объявить `extern constinit int val;`, тогда компилятор знает, что `val` уже инициализирована, а читатель кода видит, что **Initialization Order Fiasco** здесь невозможен
- `constinit` защищает от ошибки при рефакторинге: если кто-то заменит инициализатор на вызов обычной функции, код перестанет компилироваться

`if consteval` (*C++23*)

Иногда реализация для этапа компиляции и для времени выполнения должна быть разной, например, во время выполнения используются ассемблерные вставки или функции C.

```
constexpr int where() {
    if consteval {
        return 1; // compile time
    } else {
        return 2; // runtime
    }
}
```

- в C++20 для этого используется функция `std::is_constant_evaluated()` из `<type_traits>`
- `if consteval` записывается без скобок, фигурные скобки обязательны

Работа с динамической памятью

При выполнении операции выделения памяти процесс обращается к ОС, ОС находит свободные физические кадры памяти и обновляет таблицу страниц, чтобы отобразить виртуальный адрес (возвращаемый в программе) на этот физический кадр. Если физическая память заканчивается, ОС перемещает старые страницы на диск (подкачка), помечая запись,

и загружает новые. Если запрошенная по адресу переменная не находится в таблице страниц, происходит *page fault* и ОС загружает нужную страницу обратно в память. Поэтому работа с динамической памятью медленнее, чем со стеком.

Выделенная процессу память будет считаться занятой, пока с помощью операции освобождения памяти она не будет явно освобождена и отдана ОС. Если динамическую память не освободить, то она будет освобождена только по факту завершения процесса

При работе с динамической памятью могут возникать следующие проблемы:

- Утечки памяти (memory leak) - когда выделенная память не освобождается и уже не используется. Такая память будет считаться ОС занятой до тех пор, пока не завершится весь процесс (программа).
- Двойное удаление (double free) - когда производится вызов оператора освобождения памяти дважды для одного указателя, это **UB**: часто программа падает, но может и продолжить работу с повреждённой кучей. Найти такую ошибку помогает *AddressSanitizer*.
- Висячий указатель (dangling pointer) - когда указатель указывает на область памяти, которая уже была освобождена (удалена), но сам указатель не был обнулен, либо это другой указатель на ту же область памяти.

Операторы `new` и `delete`

Для выделения динамической памяти используется оператор `new`, а для освобождения оператор `delete`.

На самом деле оператор `new` обращается не напрямую к ОС, а к аллокатору стандартной библиотеки. Аллокатор хранит у себя пул памяти и выдаёт блоки из него. Только когда пул заканчивается, происходит запрос в ОС (системный вызов).

При освобождении памяти аллокатор обычно возвращает блок в свой пул, а **НЕ** в ОС. Поэтому размер памяти процесса после `delete` может не уменьшиться. Вся память процесса возвращается ОС при его завершении.

Синтаксис: `<type>* <ptr_name> = new <type>;`

Синтаксис с инициализацией:

- `<type>* <ptr_name> = new <type>(<value>);`
- `<type>* <ptr_name> = new <type>{<value>};`

```
int* ptr = new int;
int* ptr_init = new int(5);
delete ptr;
delete ptr_init;
```

Оператор `new` с аргументами (`new T(args)`) выполняет два действия:

- вызывает функцию выделения памяти заданного размера `operator new(sizeof(T))`

- вызывает конструктор типа в выделенной памяти

Оператор `delete` выполняет обратные действия:

- вызывает деструктор типа
- вызывает `operator delete(arg)`

Если память выделить не удалось, `new` бросает исключение `std::bad_alloc`.

Есть версия, не бросающая исключения, которая возвращает `nullptr`. Для её вызова используется `std::nothrow` из заголовочного файла `<new>` после `new`:

```
int* ptr = new (std::nothrow) int[1'000'000];
if (ptr != nullptr) {
    // do something
}
delete[] p;
```

ВАЖНО. `delete` и `delete[]` от `nullptr` ничего не делают, проверять указатель перед удалением **НЕ** нужно.

Операторы `new[]` и `delete[]`

Также есть соответствующая версии операторов для работы с массивами `new[]`, `delete[]`. Обязательно необходимо освобождать память с использованием парного оператора.

Синтаксис: `<type>* <name> = new <type>[<size>];`

```
int* arr = new int[10];
delete[] arr;
```

Синтаксис с инициализацией аналогичен созданию обычных массивов:

```
int* arr = new int[5]{1, 2, 3}; // {1, 2, 3, 0, 0}
int* arr_2 = new int[5]();      // {0, 0, 0, 0, 0}
int* arr_3 = new int[5]{};      // {0, 0, 0, 0, 0}
int* arr_4 = new int[5];        // {*, *, *, *, *}
```

Смешивание `new`/`delete[]` или `new[]`/`delete` — это **UB**.

Переопределение операторов `new` и `delete`

Глобальные операторы `new` и `delete`:

```
void* operator new(std::size_t);
```

```
void operator delete(void*) noexcept;
```

Операторы `new` и `delete` внутри класса позволяют перегрузить эти операторы:

```
struct Struct {
    static void* operator new(std::size_t);
    static void operator delete(void*, std::size_t) noexcept;
};
```

- вызов `new Struct` использует перегруженные операторы
- внутри класса операторы неявно `static`, поэтому можно не указывать
- версия `operator delete` со вторым параметром `std::size_t` позволяет получить размер объекта

Для вызова глобальных операторов в таком случае можно воспользоваться оператором разрешения области видимости `::operator new`, `::operator delete`:

```
static void* Struct::operator new(std::size_t bytes) {
    std::cerr << "allocate: " << bytes << " bytes\n";
    return ::operator new(bytes);
}
static void Struct::operator delete(void* ptr, std::size_t bytes) noexcept {
    std::cerr << "deallocate: " << bytes << " bytes\n";
    ::operator delete (ptr);
}

Struct* s = new Struct;   // Struct::operator new
delete s;                 // Struct::operator delete
Struct* g = ::new Struct; // глобальный ::operator new
::delete g;
```

Оператор placement `new` переопределить нельзя:

```
void* operator new(std::size_t, void*) noexcept;
```

Ключевое слово `noexcept`

Ключевое слово `noexcept` является спецификатором функции (в том числе метода, оператора) и оператором, проверяющим на этапе компиляции может ли выражение выбросить исключение `noexcept(<expression>)`.

Спецификатор `noexcept` пишется после сигнатуры функции и перед `{`. Спецификатор сообщает компилятору, что код внутри не выбрасывает исключений, и позволяет ему выполнить оптимизации (убрать код для раскрутки стека).

Если исключение всё же будет выброшено из помеченной функции, то вызовется `std::terminate`, который завершит процесс.

Хорошим тоном считается писать `noexcept` там, где действительно код не может выбросить исключение.

Правила использования:

- Деструкторы по умолчанию `noexcept`
- Функции освобождения памяти (`free` , `delete`) должны быть `noexcept`
- Функции обмена (`swap`) должны быть `noexcept`
- Хорошими кандидатами на использование `noexcept` являются простые `const` методы
- Move-операции должны быть `noexcept` , когда это возможно
- **НЕ** следует использовать `noexcept(false)` без веской причины

Категория выражений (value categories)

Помимо типа каждое выражение в C++ характеризуется категорией (value categories).

Классификация до C++11:

- *lvalue* - имеет адрес, может быть слева от присваивания `=`
- *rvalue* - временное значение, справа от присваивания `=`

Современная классификация (C++11) построена на двух свойствах:

- Идентичность (I) - параметр, по которому можно понять, ссылаются ли два выражения на один объект (например, наличие адреса в памяти)
- Перемещаемость (M) - возможно ли переместить объект, отдать владение ресурсом

В основу классификации легли три первичные категории:

- *lvalue* (locator/left value)
- *xvalue* (expiring value) - объект, ресурсы которого можно забрать (с истекающим временем жизни): результат материализации временного объекта, `std::move(x)` , вызова функции, возвращающей `T&&`
- *prvalue* (pure rvalue)

```
        glvalue(I)      rvalue(M)
         /     \        /      \
lvalue(I~M)   xvalue(IM)     prvalue(~IM)
```

---

rvalue-ссылка

В C++11 вводится rvalue-ссылка `&&` .

Основные отличия rvalue-ссылки `&&` от lvalue-ссылки `&` :

- можно проинициализировать только rvalue выражением

- при возврате из функции `&&`, возвращается rvalue выражение
- продлевает жизнь временным объектам (как и `const T&`)

Во всем остальном ведет себя также как lvalue-ссылка. Сам по себе идентификатор, объявленный rvalue-ссылкой, является lvalue.

```
int l = 18;
int&& r = l; // compilation error (l is not rvalue)
int&& x = 18; // ok
int&& y = x; //compilation error (x is not rvalue)
int&& m = std::move(l); // ok
int&& sc = static_cast<int&&>(x); // ok
m = 0; // ok l = 0, m = 0
```

В языке присутствует базовое неявное приведение *lvalue-to-prvalue*:

```
int x, y;
x = x + 1;        // lvalue = prvalue
y = x;            // lvalue = lvalue to prvalue
y = std::move(x); // lvalue = xvalue
```

Связывание ссылок

Rvalue-ссылка **НЕ** может быть связана с **lvalue**. Lvalue-ссылка **НЕ** может быть связана с **rvalue**.

```
int x = 1;
int&& y = x * 3; //ok
int&& b = x;     //fail, not rvalue

int& c = x * 3;       // fail, not lvalue;
const int& d = x * 3; //ok, lifetime prolongation for const ref

int&& e = y; // fail, not rvalue
int& f = y;  // ok
```

Перегрузка функции по ссылке

Функция может быть перегружена по типу принимаемой ссылки:

```
int foo(int& v); // 1
int foo(int&& v); // 2
int foo(const int& v); // 3
int foo(const int&& v); // 4
```

```
int x = 1;

foo(x); // -> 1
foo(1); // -> 2

const int y = 1;

foo(y); // -> 3
foo(std::move(y)); // -> 4
```

Квалификация метода по виду ссылки (*C++11*)

Метод может быть перегружен, в зависимости от того вызывается он для **lvalue** или **rvalue** ссылки.

Для этого после закрывающей скобки аргументов добавляется квалификатор **&** или **&&** Таким образом, можно определить различное поведение при вызове функции от временного объекта (rvalue) `&&` и постоянного (lvalue) `&`

```
struct S {
    int foo() &;
    int foo() &&;
};
```

Оператор присваивания можно пометить одним амперсандом `&` и не создавать версию с `&&`, чтобы к временному объекту нельзя было присваивать. Тогда будет недоступно присваивание к временному объекту, что достаточно логично:

```
struct S {
    S() = default;
    S& operator=(const S& other) &;
};

S a;
S() = a; // compile error
```

Temporary materialization (C++17)

Материализация временного объекта (*temporary materialization*) представляет собой неявное преобразование из *prvalue* в *xvalue*.

Материализация временного объекта происходит:

- при связывании ссылки с чисто временным объектом (категории prvalue)
- при доступе к нестатическому полю класса prvalue-объекта

- при вызове нестатических методов от prvalue объекта (поскольку в метод неявно передается указатель `this`)
- при выполнении преобразования array-to-pointer или обращению по индексу `[]` к prvalue массиву
- при инициализации объекта `std::initializer_list<T>` посредством списка инициализации в фигурных скобках
- когда prvalue появляется в отбрасывающем результат выражении (игнорируем результат prvalue выражения, возвращающего тип какого-либо класса)

1. Связывание ссылки с чисто временным объектом

```
const int& r1 = 10; // prvalue 10 is materialized to get cref
int&& r2 = 5 + 3;   // prvalue from expr '5+3' is materialized to get &&

void foo(const std::string&);
foo("hello"); // std::string materializes from "hello" to get cref
```

2. Доступ к нестатическому полю класса, prvalue объекта

```
struct Point { int x = 0, int y = 0};
int val = Point{3, 4}.x; // Point{3,4} is materialized to access .x
```

3. Вызов нестатического метода от rvalue объекта

```
struct Logger {
    void log() const { std::cout << "Logging\n"; }
};
Logger().log(); // Logger() is materialized to call log(), need this.
std::string("hello").length(); // the same case
```

4. Обращение к prvalue массиву `[]` или при преобразовании array-to-pointer

```
void bar(const int*);
bar((int[]){10, 20});

int* p1 = (int[3]){1, 2, 3};.
int val = ((int[]){10, 20, 30})[1];
```

5. Инициализация объекта `std::initializer_list<T>`

```
#include <initializer_list>
void func(std::initializer_list<int> lst) {}
func({1, 2, 3});
```

6. Когда prvalue в discarded-value expression

```
struct Widget {
    Widget() { std::cout << "Created\n"; }
    ~Widget() { std::cout << "Destroyed\n"; }
};
Widget(); // materialized to the end of expression ;
```

Материализации временного объекта **НЕ** возникает, когда используется prvalue того же типа (при `direct-initialization` или `copy-initialization`). Такие объекты создаются напрямую из инициализирующего prvalue. Это является гарантированным *copy elision*, начиная с C++17.

Семантика перемещения

Чтобы избежать излишнего копирования, когда создается объект из объекта, который больше не нужен, в языке реализована семантика перемещения (move-семантика).

Семантика перемещения позволяет забрать у другого объекта ресурсы, которыми он владеет. Причем, объект, у которого забрали данные, должен остаться в консистентном состоянии, то есть после перемещения из него данных, его состояние должно было корректным (часто дефолтным, нулевым) и им можно было продолжать пользоваться, например, присвоив ему нового значения.

Начиная с C++11 у класса появляется перемещающий конструктор и перемещающий оператор присваивания.

```
Class(Class&& other);
Class& operator=(Class&& other);
```

Компилятор генерирует move-конструктор и move оператор присваивания, вызывая соответствующие move операции для полей класса.

Для скалярных типов (примитивные типы, указатели) перемещающие методы выполняют **копирование**. Поэтому сгенерированный компилятором move-конструктор класса с сырым указателем **копирует** указатель, и деструкторы двух объектов освободят одну память.

Владение ресурсом подразумевает, например, указатель на динамическую память внутри класса и выделение памяти в конструкторе и освобождение в деструкторе. Пусть экземпляр класса создается на стеке, а данные в динамической памяти. Копирование по умолчанию подразумевает копирование значений всех полей, в случае указателя скопируются не данные, а указатель (адрес на выделенную память). В такой ситуации возникает проблема двойного удаления по указателю, так как два объекта имеют одинаковые указатели на одну память и при выходе из области видимости объектов в деструкторе будет двойное освобождение памяти по указателю.

Для такого класса определяется конструктор копирования и оператор присваивания копированием, чтобы данные в динамической памяти действительно копировались:

```
class String {
public:
    String(size_t size) : size_(size), cap_(size), data_(new char[size + 1]) {
        std::fill_n(data, size + 1, '\0');
    }
    String(const String& other) :
        size_(other.size_),
        cap_(other.size_),
        data_(new char[other.size_ + 1]) {
        std::strcpy(data_, other.data_);
    }
    String& operator=(const String& other) {
        if (this != &other) {
            delete[] data;
            size = other.size;
            cap_ = other.size;
            data_ = new char[size + 1];
            std::strcpy(data_, other.data_);
        }
        return *this;
    }
    ~String() { delete[] data_; }

    size_t cap_;
    size_t size_;
    char* data_;
};
```

Когда объект больше не нужен, а его данные нужны, или как минимум нужна выделенная память, чтобы не выделять её снова, можно переместить ресурс (память, файловый дескриптор, сокет и т.д.) из одного объекта в другой, а не копировать их. Это обеспечивает значительный прирост производительности, особенно для объектов, владеющих "тяжелыми" ресурсами.

Но объект, из которого забирают ресурс, должен остаться в консистентном состоянии и его можно было использовать:

```
String(String&& other) noexcept
    : cap_(other.cap_), size_(other.size_), data_(other.data_) {
    // необходимо оставить other в валидном состоянии
    other.cap_ = 0;
    other.size_ = 0;
    other.data_ = nullptr;
}
String& operator=(String&& other) noexcept {
    if (this != &other) {
        // очищаем свои данные
        delete[] data_;
        // копируем
        cap_ = other.cap_;
        size_ = other.size_;
```

```
        // забираем данные
        data_ = other.data_;
        // необходимо оставить other в валидном состоянии
        other.cap_ = 0;
        other.size_ = 0;
        other.data_ = nullptr;
    }
    return *this;
}
```

Вызвать данные конструкторы можно следующим образом:

```
String str(1000);
auto str_move = std::move(str); // call move ctor

String str_other(10);
String str_moved(100);
str_moved = std::move(str_other); // call move assignment
```

Функция `std::move`

Функция `std::move` необходима для того, чтобы при вызове функции от объекта перенаправить компилятор на использование версии с rvalue-ссылкой.

Функция действует на этапе компиляции и всё, что она делает, это возвращает rvalue-ссылку на переданный ей объект.

```
std::string str = "I will be moved";
auto str_copy = str;
auto str_moved = std::move(str); // call move ctor for string
std::cout << str_copy == str_moved << std::endl; // 1 (true)
std::cout << str.empty() << std::endl; // 1 (true)
```

Функция `std::move` от `const` объекта даёт `const T&&`, и вызывается копирующий конструктор.

Правило нуля (*Rule of Zero*)

"Правило нуля" - принцип программирования на C++, который гласит, что класс **НЕ** должен определять ни один из пяти специальных методов, если их можно автоматически сгенерировать компилятором:

- деструктор
- конструктор копирования
- оператор присваивания копированием
- конструктор перемещения

- оператор присваивания перемещением

Если ваш класс **НЕ** управляет ресурсами (памятью), а его данные — это простые типы или классы, которые сами управляют ресурсами (например, `std::string`, `std::vector`, умные указатели), то вам **НЕ** нужно писать свои специальные методы.

Класс **НЕ** должен самостоятельно управлять ресурсами, если это не класс управления ресурсами.

Правило пяти (*Rule of Five*)

"Правило пяти" - принцип программирования на C++, который гласит, что если пользовательский класс управляет ресурсами (память, файловые дескрипторы) и определяет хотя бы один из пяти специальных методов, то он **должен явно определить все пять**:

- деструктор
- конструктор копирования
- оператор присваивания копированием
- конструктор перемещения
- оператор присваивания перемещением

Такой подход позволяет избежать ошибок управления памятью (висячие указатели, двойное удаление).

До возникновения семантики перемещения в C++11 использовали "Правило трёх".

Правила для специальных методов

Компилятор генерирует специальные методы самостоятельно в зависимости от условий.

Специальные методы класса:

- конструктор по умолчанию
- деструктор
- конструктор копирования
- оператор присваивания копированием
- конструктор перемещения
- оператор присваивания перемещением

Конструктор по умолчанию создается:

- default, если пользователь не объявил ни одного конструктора, в том числе конструктора копирования или перемещения

Деструктор по умолчанию создается:

- default, если нет ни одного пользовательского деструктора

Конструктор копирования (аналогично копирующее присваивание):

- `= delete`, если объявлен move-конструктор или move-присваивание
- иначе default, если его нет (но если при этом есть деструктор или парный копирующий метод, то deprecated)

Перемещающий конструктор (аналогично перемещающее присваивание):

- отсутствует, если его нет, но при этом есть парный перемещающий метод или copy ctor, или copy operator=, или dtor (при наследовании если деструктор объявлен virtual)
- иначе default, если его нет

Copy Elision

Copy elision (пропуск копирования) в C++ — это оптимизация компилятора, устраняющая создание временных объектов при возврате из функций или передаче по значению, что исключает вызов конструкторов копирования/перемещения.

Начиная с C++17 компилятор обязан применять *copy elision* при инициализации объекта результатом функции, возвращающей тот же тип, что повышает производительность и делает код более эффективным.

Оптимизация позволяет не тратить время на копирование/перемещение, а деструкторы для временных объектов не вызываются.

Return Value Optimization (RVO)

Return Value Optimization (RVO) - оптимизация компилятора, позволяющая не создавать локальный объект, который используется в качестве возвращаемого значения, а конструировать возвращаемый объект на месте вызова функции. Такая оптимизация позволяет устранить лишний вызов конструктора копирования или перемещения.

RVO применима, если возвращаемое значение prvalue-выражение, то есть временный объект.

Начиная с *C++17* RVO - это не оптимизация, а правило, которое должно выполняться компилятором.

Named Return Value Optimization (NRVO)

Named Return Value Optimization (NRVO) - оптимизация компилятора, аналогичная RVO, но действующая для локального lvalue-объекта, имеющего имя, и возвращаемого из функции.

На месте вызова функции вставляется инициализация объекта, принимающего возвращаемое значение функции. В аргументы функции добавляется указатель на данный объект и все вычисления возвращаемого значения выполняются над объектом под указателем.

NRVO может быть применена только когда тип возвращаемого объекта и тип возвращаемого значения сигнатуры функции точно совпадают без учёта cv-квалификаторов, и объект — локальная переменная, а **НЕ** параметр функции.

По этой причине **НЕ** следует возвращаемое значение оборачивать в `std::move()`, так как тип возвращаемого значения не будет совпадать и, следовательно, оптимизация не будет применена.

```
std::string make_str() {
    std::string s(100, 'x');
    return std::move(s);     // warning: -Wpessimizing-move, мешает NRVO
}
std::string get_param(std::string s) {
    return std::move(s);     // warning: -Wredundant-move, move и так неявный
}
```

- GCC и Clang выдают эти предупреждения с `-Wall` `-Wextra`

Если NRVO не применена (например, в разных ветках возвращаются разные объекты), возвращаемая локальная переменная или параметр функции всё равно рассматривается как rvalue (implicit move, C++11, расширено в C++20 и C++23). Поэтому возвращение локальной переменной из функции - это в худшем случае перемещение, но **НЕ** копирование.